

## Capstone Project 3: Financial "Zero-Trust" Ecosystem

**Focus:** Service Mesh (Istio), Event-Driven Architecture, and Compliance.

### **Problem Statement**

A high-security Fintech firm requires:

- **East-West Security:** All communication between internal microservices must be encrypted.
- **Strict Authorization:** The "Rewards" service should never be able to talk to the "Credit Card" DB.
- **Auditability:** Every financial transaction must be logged to a tamper-proof location.
- **Decoupling:** High-latency banking checks should not slow down the user experience.

**Goal:** Implement Anthos Service Mesh (ASM) and Pub/Sub for a secure, asynchronous microservices environment.

### **1. ARCHITECTURE OVERVIEW (WHAT YOU ARE BUILDING)**

You are designing a **Zero-Trust FinTech Platform** with:

- **Service Mesh** → secure service-to-service communication
- **Event-Driven Architecture** → decoupled processing
- **Compliance Layer** → audit + traceability

## Final Architecture Flow

User → API Gateway → Istio Ingress Gateway

↓

Microservices (Service Mesh)

↓

Pub/Sub (Async Processing)

↓

Consumers (Fraud, Audit, Notifications)

↓

Cloud Storage / BigQuery (Audit Logs)

---

## 2. CORE CONCEPT: ZERO-TRUST MODEL

---

### What is Zero Trust?

Traditional:

- Trust based on network (IP-based)

Zero Trust:

- Trust based on **identity + policy**
- 

### Principles

1. Never trust by default
2. Always verify identity
3. Encrypt everything
4. Enforce least privilege

---

### **3. SERVICE MESH (ISTIO / ANTHOS SERVICE MESH)**

---

#### **3.1 What is a Service Mesh?**

A layer that:

- Controls communication between microservices
  - Adds:
    - Security
    - Observability
    - Traffic control
- 

#### **3.2 Architecture Components**

##### **1. Data Plane**

- Envoy sidecar proxies
- Attached to each pod

Flow:

Service A → Envoy → Envoy → Service B

---

##### **2. Control Plane (Istio)**

- Manages:
    - Policies
    - Certificates
    - Routing rules
-

### 3.3 Sidecar Injection

---

#### Concept

Automatically attach proxy to each pod

---

#### Command

kubectl label namespace prod istio-injection=enabled

---

#### What Happens Internally

- Pod gets:
    - App container
    - Envoy proxy
- 

#### Benefit

- No code changes required
- 

### 4. CASE STUDY BREAKDOWN (DEEP)

---

#### Case 1: Service Mesh Integration

---

##### Problem

Managing TLS certificates manually across 50+ services

---

##### Solution

- ASM auto-generates certificates
  - Rotates them automatically
- 

### **Internal Flow**

- Istio CA issues certs
  - Envoy uses them for encryption
- 

### **Outcome**

- No manual certificate management
- 

### **Case 2: Mutual TLS (mTLS)**

---

#### **Concept**

Both client and server authenticate each other

---

#### **Modes**

- PERMISSIVE (mixed)
  - STRICT (recommended)
- 

#### **Implementation**

apiVersion: security.istio.io/v1beta1

kind: PeerAuthentication

metadata:

name: default

spec:

mtls:

mode: STRICT

---

### **What Happens**

- All traffic encrypted
  - Identity verified via certificates
- 

### **Outcome**

- Prevents packet sniffing
  - Enforces encrypted communication
- 

## **Case 3: Fine-Grained Access Control**

---

### **Problem**

Unauthorized service access

---

### **Solution**

Istio AuthorizationPolicy

---

### **Example**

apiVersion: security.istio.io/v1beta1

kind: AuthorizationPolicy

metadata:

name: allow-gateway

spec:

rules:

- from:

- source:

principals: ["cluster.local/ns/prod/sa/gateway"]

to:

- operation:

methods: ["POST"]

---

### Internal Logic

- Identity-based access (not IP)
  - Checks:
    - Who (service account)
    - What action (HTTP method)
- 

### Outcome

- Strict service boundaries
- 

### Case 4: Event-Driven Architecture (Pub/Sub)

---

#### Problem

Synchronous banking APIs slow down system

---

## **Solution**

Decouple using Pub/Sub

---

## **Flow**

Frontend → Publish → Topic

↓

Multiple Subscribers

---

## **Mini Project**

### **Publish**

```
from google.cloud import pubsub_v1

publisher = pubsub_v1.PublisherClient()

topic_path = publisher.topic_path("PROJECT_ID", "transaction-topic")

publisher.publish(topic_path, b"Transaction Data")
```

---

### **Subscribe**

```
from google.cloud import pubsub_v1

def callback(message):

    print(message.data)

    message.ack()
```



```
subscriber = pubsub_v1.SubscriberClient()
```

```
subscription_path = subscriber.subscription_path("PROJECT_ID", "sub")
```

```
subscriber.subscribe(subscription_path, callback=callback)
```

---

### **Outcome**

- Loose coupling
  - Better scalability
- 

### **Case 5: Event-Driven Auditing**

---

#### **Problem**

Need tamper-proof audit logs

---

#### **Solution**

- Pub/Sub → Cloud Function → Storage
- 

#### **Flow**

Transaction → Pub/Sub → Cloud Function → Storage Bucket

---

### **Mini Project**

```
def audit_log(event, context):
```

```
    import base64
```

```
    data = base64.b64decode(event['data']).decode('utf-8')
```

```
with open("/tmp/log.txt", "a") as f:
```

```
    f.write(data)
```

---

## Outcome

- Immutable logs
  - Compliance ready
- 

## Case 6: Canary Releases

---

### Concept

Deploy new version gradually

---

### Istio VirtualService

apiVersion: networking.istio.io/v1beta1

kind: VirtualService

spec:

http:

- route:

- destination:

host: service

subset: v1

weight: 99

- destination:

host: service

subset: v2

weight: 1

---

## Outcome

- Safe deployments
  - Easy rollback
- 

## Case 7: Circuit Breaking

---

### Problem

Failing service affects entire system

---

### Solution

apiVersion: networking.istio.io/v1beta1

kind: DestinationRule

spec:

trafficPolicy:

outlierDetection:

consecutive5xxErrors: 5

interval: 10s

baseEjectionTime: 30s

---

## Internal Working

- Detect failures
  - Remove bad pods temporarily
- 

### **Outcome**

- System stability
- 

### **Case 8: Centralized Secrets**

---

#### **Solution**

- Secret Manager
  - Workload Identity
- 

#### **Flow**

Pod → IAM → Secret Manager

---

### **Outcome**

- No secrets in code
  - Secure access
- 

### **Case 9: End-to-End Tracing**

---

#### **Problem**

Difficult debugging

---

## **Solution**

Cloud Trace + Istio

---

## **Flow**

Request → Service A → Service B → Service C

Trace shows:

- Latency at each step
- 

## **Outcome**

- Root cause analysis
- 

## **Case 10: Compliance Dashboard**

---

## **Solution**

Anthos Service Mesh UI

Shows:

- Traffic flow
  - Encryption status
  - Service dependencies
- 

## **FINAL SYSTEM CHARACTERISTICS**

---

## **Security**

- mTLS everywhere

- Identity-based access
- 

## **Scalability**

- Async processing via Pub/Sub
- 

## **Reliability**

- Circuit breakers
  - Retry policies
- 

## **Observability**

- Tracing + logging
- 

## **REAL FINTECH IMPACT**

---

### **Before**

- Hardcoded secrets
  - Synchronous failures
  - No visibility
- 

### **After**

- Zero-trust architecture
- Fully auditable system
- Resilient microservices